Forecasting California Retail Gasoline Prices: A Comparative Study of ARIMAX and Recurrent Neural Network Architectures

---

STAT 426 — Written Report
Professor George Michailidis
University of California, Los Angeles
March 2026

Team Members: Antoinette Nguyen, Nick Wong, Kathleen Wu

Abstract

Accurate forecasting of retail gasoline prices is critical for consumers, policymakers, and businesses who must plan around fuel cost volatility. This study presents a comparative analysis of a classical time-series model and modern deep learning models for weekly retail gasoline price forecasting. We employed a multivariate time series data of 1343 weekly observations as the basis for this analysis. We considered time series between May 2000 to February 2026, incorporating five macroeconomic and energy-market covariates sourced from the Energy Information Administration (EIA) and the Federal Reserve Economic Data (FRED) database: WTI crude oil, PADD5 gasoline inventories, US refinery utilization rate, US product supplied of finished motor gasoline, data-engineered seasonal encodings. Our baseline classical model is an ARIMA-X(1,1,1), specification selected through grid search, evaluated using one-step-ahead rolling forecast on a test set. For deep learning models, we implemented three recurrent neural network (RNN) variants, Long Short Term Memory (LSTM), Gated Recurrent Unit (GRU), and a Vanilla RNN all trained with an identical two-layer, 128 hidden unit using the Adam optimizer with early stopping. On the 135 week test set, ARIMA-X achieves the lowest RMSE of \$0.0701/gallon ($R^2 = 0.9616$), followed closely by the GRU at \$0.0864 ($R^2 = 0.9417$), while the LSTM falls behind at \$0.1876. A GRU ablation study across various hidden sizes, dropout rates, and lookback windows reveal that a 12 week lookback window with 128 hidden units and no dropout yields the most optimal RMSE. The results reveal that while deep architectures capture nonlinear dynamic, the classical ARIMA-X baseline combined with covariates remain highly competitive for energy price forecasting.

Contents

1 Introduction & Motivation **4**
1.1 Problem Statement . . . 4
1.2 Real-World Motivation . . . 4
1.3 Objectives & Success Metrics . . . 4

2 Related Work **4**

3 Dataset and Exploratory Data Analysis **5**
3.1 Data Sources . . . 5
3.2 Descriptive Statistics . . . 6
3.3 Correlation Analysis . . . 6
3.4 Data Preprocessing . . . 7

4 Methodology & Architectures **8**
4.1 Baseline Model: ARIMAX . . . 8
4.1.1 Model Specification . . . 8
4.1.2 Order Selection . . . 8
4.1.3 Rolling Evaluation Protocol . . . 9
4.2 Advanced Models: Recurrent Neural Networks . . . 10
4.2.1 Vanilla RNN . . . 10
4.2.2 Long Short-Term Memory (LSTM) . . . 10
4.2.3 Gated Recurrent Unit (GRU) . . . 10
4.3 Architecture Summary . . . 11
4.4 Optimization & Training . . . 11

5 Experimental Setup **12**
5.1 Computational Times: . . . 12
5.1.1 ARIMAX Training Time . . . 12
5.1.2 Total Project Compute . . . 12
5.2 Hyperparameters . . . 13
5.3 Reproducibility . . . 13

6 Results **13**
6.1 Model Performance Comparison . . . 13
6.2 Learning Curves . . . 14
6.3 Forecast Visualization . . . 14
6.4 Bar Chart Comparison . . . 15
6.5 ARIMAX Residual Diagnostics . . . 15

7 Discussion, Ablations, & Error Analysis **16**
7.1 Discussion: Why Does ARIMAX Outperform Deep Learning? . . . 16
7.2 Ablation Study: GRU Architecture Sensitivity . . . 17
7.2.1 Hidden Size . . . 17
7.2.2 Dropout Rate . . . 17
7.2.3 Lookback Window . . . 17
7.3 Error Analysis . . . 18
7.3.1 Sources of Error . . . 18

7.4 Interpretation . . . 18

8 Conclusion & Future Work **18**
8.1 Summary of Findings . . . 18
8.2 Limitations . . . 19
8.3 Future Work . . . 19
8.3.1 Extended Study: Further Timeline, Larger Computational Resources, and Adaptation for a Thesis . . . 19

Team Contributions **21**

Codebase **21**

List of Tables

1 Data sources and variable definitions. . . . 5
2 Descriptive statistics (full sample). . . . 6
3 Pearson correlation matrix. . . . 6
4 Chronological data split. . . . 7
5 ARIMAX order search results (validation). . . . 9
6 Shared RNN architecture across variants. . . . 11
7 Hardware and software configuration. . . . 12
8 RNN training times (CPU). . . . 12
9 Final hyperparameter settings. . . . 13
10 Test performance comparison. . . . 13
11 ARIMAX residual summary statistics. . . . 16
12 GRU ablation: hidden size. . . . 17
13 GRU ablation: dropout rate. . . . 17
14 GRU ablation: lookback window length. . . . 17
15 Final test-step predictions. . . . 18

List of Figures

1 Five-panel time-series visualization of predictors (2000–2026). . . . 7
2 Train/Validation/Test split visualization for California gas prices. . . . 8
3 ACF and PACF of the differenced training gas price series with 95% confidence bands. 9
4 Training and validation loss curves for GRU, LSTM, and Vanilla RNN (no dropout). 14
5 Model forecasts versus actual California gas prices on the test period (135 weeks). . 15
6 RMSE and MAE comparison across models. . . . 15
7 ARIMAX residual diagnostics: . . . 15
8 ACF and PACF of ARIMAX residuals (50 lags) with 95% confidence bands. . . . 16
9 GRU ablation study summary table: rolling one-step-ahead test performance. . . . 18

1 Introduction & Motivation

1.1 Problem Statement

This study addresses the problem of retail gas price forecasting for the state of California. California consistently maintains the highest gasoline price among all the U.S. states, driven by state-specific environmental regulations, and limited refinery capacity, geographically isolation from Gulf Coast refineries, and cap-and-trade carbon pricing.

1.2 Real-World Motivation

Gasoline expenditures represent one of the largest household spending categories in the United States, and gasoline remains the primary transportation fuel used by most Americans. According to the Energy Information Administration (EIA), California ranked as the second-largest fuel-consuming state as of 2025 [6]. In a state with roughly 27 million registered vehicles, even modest improvements in forecasting accuracy can meaningfully inform economic decisions and benefit multiple stakeholders.

- **Consumer and businesses** benefit from making informed decisions with regards to budgeting and spending.
- **Policymakers** often use gasoline prices to evaluate household welfare and allow government to intervene in the energy market.
- **Transportation and logistic companies** rely on fuel cost projections for pricing and fleet management.

Recent global events, including the COVID-19 pandemic in 2020, the Russia–Ukraine conflict in 2022, have introduced volatility into the global energy market, thus making accurate and reliable forecasting much more difficult. Our study uses data from May 2000 through February 2026, spanning multiple economic events and periods of major disruption in the energy market.

1.3 Objectives & Success Metrics

Our objective is to compare the predictive performance of a classical time series model (ARIMA-X) against three recurrent neural network architectures on a held-out test set using a Rolling 1-step ahead forecasting approach. We evaluated all models using the following metrics:

1. **Root Mean Squared Error (RMSE):** Our primary metric, measuring average forecast deviation in dollars per gallon. RMSE penalizes large errors quadratically, making it particularly appropriate for risk-sensitive applications.
2. **Mean Absolute Error (MAE):** A robust measure of average absolute deviation, less sensitive to outliers than RMSE.
3. **Coefficient of Determination (R^2):** Measures the proportion of variance in actual prices explained by model predictions. Values near 1.0 indicate strong explanatory power.

2 Related Work

Time-series forecasting of energy commodity prices has been widely studied within the statistical modeling and data science communities, receiving extensive attention in the academic literature. Our methodological approach is designed to align with the concepts established in prior research.

Classical Econometric Approaches. ARIMA models and their extensions (ARIMAX, SARIMAX) remain foundational tools for energy price forecasting. Baumeister and Kilian (2015)[1] showed that simple autoregressive models with crude oil price predictors can outperform more complex architectures for real-time fuel price forecasting. Their work highlights the importance of parsimonious model selection and proper out-of-sample evaluation—principles we adopt through using grid search selection.

Forecasting Competitions and Benchmark Models. The M-competitions organized by Makridakis et al.[5] demonstrated that classical statistical approaches such as exponential smoothing and ARIMA-based models remain highly competitive benchmarks across a wide range of time-series forecasting tasks. These findings motivate the usage of ARIMAX as a strong baseline in our study and evaluation approach using a rolling one-step-ahead technique.

Recurrent Neural Networks for Time Series. The application of RNNs to energy time series has grown rapidly since the introduction of LSTM networks by Hochreiter and Schmidhuber (1997)[4]. LSTMs address the vanishing gradient problem that limits standard RNNs' ability to learn long-range dependencies. Cho et al. (2014) [3]introduced the Gated Recurrent Unit (GRU) as a computationally lighter alternative to LSTM, demonstrating comparable or superior performance with fewer parameters. Our comparison of LSTM (201,345 parameters) vs. GRU (151,041 parameters) versus Vanilla RNN (50,433 parameters) directly tests whether gating mechanisms justify their additional computational cost.

3 Dataset and Exploratory Data Analysis

3.1 Data Sources

We construct a multivariate weekly time-series dataset by merging five publicly available series from the U.S. Energy Information Administration (EIA) [8]and the Federal Reserve Economic Data (FRED) database [7]:

Table 1: Data sources and variable definitions.

Variable	Source	Description	Units
`gas_price`	EIA	California All Grades All Formulations Retail Gasoline Price	$/gallon
`wti_price`	FRED	West Texas Intermediate Crude Oil Spot Price	$/barrel
`padd5_inventory`	EIA	West Coast (PADD 5) Ending Stocks of Total Gasoline	Thousand barrels
`refinery_util`	EIA (WPULEUS3)	U.S. Percent Utilization of Refinery Operable Capacity	Percent (%)
`product_supplied`	EIA	Weekly U.S. Product Supplied of Finished Motor Gasoline	Thousand barrels/day

All series data are reported at a weekly frequency. The raw data was extracted from EIA and FRED website. Some data reported a certain day like Monday vs. Friday basis so they were merged using nearest-date matching with a tolerance of 7 days. The merged series data were clipped to

a common start date of **May 22, 2000** and dropping rows with missing values, the final merged dataset contains **1,343 weekly observations** spanning May 22, 2000 through February 9, 2026, a period of nearly 25 years.

3.2 Descriptive Statistics

Table 2 presents summary statistics for all five variables across the full sample.

Table 2: Descriptive statistics (full sample).

Statistic	**Gas Price** ($/gal)	**WTI Price** ($/bbl)	**PADD 5 Inv.** (k bbl)	**Refinery Util.** (%)	**Product Supplied** (k bbl/day)
Mean	3.3212	64.2346	29,687	89.03	8,955.94
Std. Dev.	1.0325	24.7072	2,085	4.94	435.88
Min	1.1460	3.3200	24,088	56.00	5,329.25
25th Pctl	2.6085	45.5100	28,203	86.35	8,721.50
Median	3.2610	63.2600	29,546	89.70	9,000.50
75th Pctl	4.0660	81.7900	31,040	92.60	9,220.63
Max	6.3640	142.5200	36,167	98.10	9,777.25

California gas prices exhibit substantial variation, ranging from a low of $1.15/gallon (late 2001, post-dot-com recession) to a peak of $6.36/gallon (mid-2022, following the Russia–Ukraine conflict and California-specific refinery disruptions). The standard deviation of $1.03 represents approximately 31% of the mean, indicating considerable volatility.

3.3 Correlation Analysis

The Pearson correlation matrix reveals the relationships shown in Table 3.

Table 3: Pearson correlation matrix.

	Gas Price	**WTI**	**PADD 5 Inv.**	**Refinery Util.**	**Product Supplied**
Gas Price	1.000	**0.763**	-0.184	-0.052	0.013
WTI	0.763	1.000	-0.103	-0.145	0.108
PADD 5 Inv.	-0.184	-0.103	1.000	-0.106	-0.251
Refinery Util.	-0.052	-0.145	-0.106	1.000	**0.448**
Product Supplied	0.013	0.108	-0.251	0.448	1.000

Retail gasoline prices and crude oil prices (WTI) exhibit a strong positive association ($r = 0.763$), indicating that crude oil remains a key driver of retail gasoline price dynamics. Gasoline inventories show a weak negative relationship with retail gasoline prices ($r = -0.184$), suggesting that higher inventory levels may exert mild downward pressure on prices. In contrast, refinery utilization and product supplied display very weak correlations with retail gasoline prices. However, refinery utilization and product supplied exhibit a moderate positive relationship ($r = 0.448$), reflecting the connection between refinery production activity and gasoline supply.

3.4 Data Preprocessing

Seasonal Encoding. Rather than using raw month indicators (which introduce artificial discontinuities between December and January), we encode seasonality using sinusoidal transformations of the month index $m \in \{0, 1, \ldots, 11\}$ [2]: $\text{month_cos} = \cos(2\pi m/12)$ and $\text{month_sin} = \sin(2\pi m/12)$. This produces two smooth, continuous features that together uniquely identify each month while respecting the cyclical nature of seasonal effects. These features are used exclusively in the ARIMAX model as exogenous predictors.

Normalization. All five numeric features are scaled to the $[0, 1]$ range using `MinMaxScaler`, fit exclusively on the training set to prevent data leakage. The same transformation parameters are then applied to the validation and test sets.

Train/Validation/Test Split. We adopt a chronological train/validation/test split before normalizing the data:

Table 4: Chronological data split.

Split	Observations	Proportion	Date Range
Train	1,074	80.0%	May 2000 – ∼June 2021
Validation	134	10.0%	∼June 2021 – ∼Dec 2023
Test	135	10.1%	∼Dec 2023 – Feb 2026

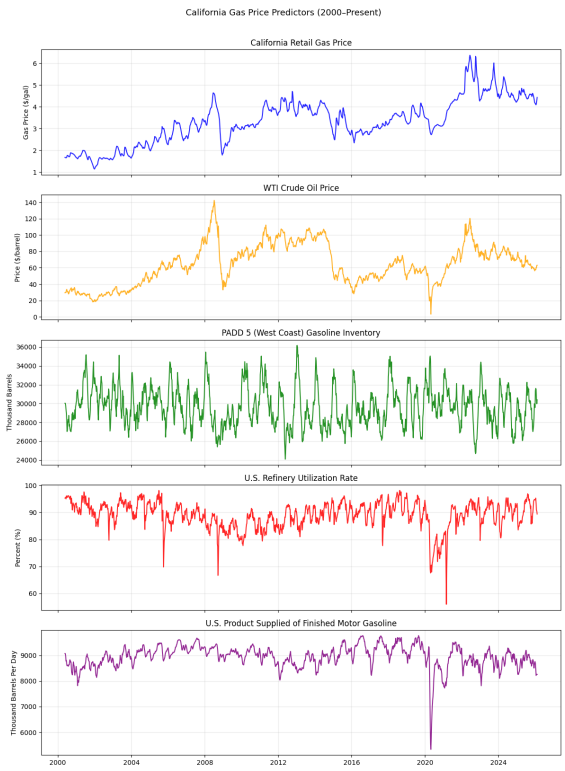

Figure 1: Five-panel time-series visualization of predictors (2000–2026).

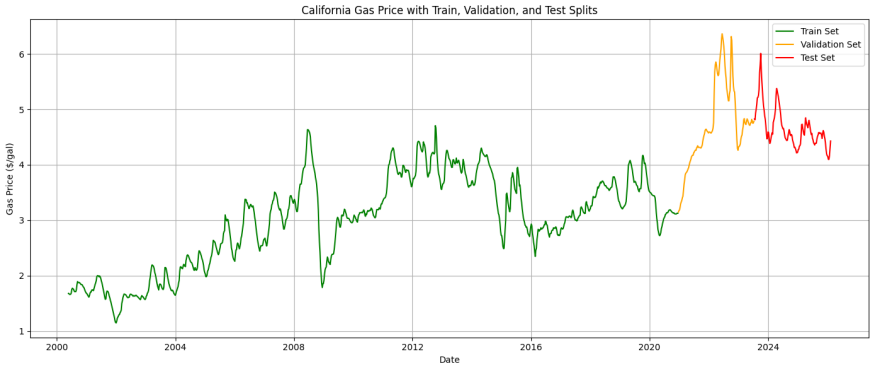

Figure 2: Train/Validation/Test split visualization for California gas prices.

4 Methodology & Architectures

This section presents the mathematical concepts of the modeling techniques that were used. We begin with the ARIMAX baseline, then the recurrent neural network variants, and conclude with optimization and training procedures.

4.1 Baseline Model: ARIMAX

4.1.1 Model Specification

Our baseline is an **Autoregressive Integrated Moving Average with Exogenous Predictors (ARIMAX)** model, which is an extension to the classical ARIMA framework by incorporating external predictor variables. ARIMAX combines autoregressive (AR), integrated (I), and moving average (MA), in addition to exogenous Variables (X). These can be external factors not part of the time series but can have an effect on it, which thus can provide a more comprehensive analysis and forecast. The model is typically is denoted as ARIMAX(p, d, q) where the 'p' denotes the autoregressive order, which is the number of lagged observations of the dependent variable, 'd' representing the differencing order and indicating how many times the time series must be differenced to achieve stationarity, and 'q' signifying the moving average order, which is the number of lagged forecast errors in the model.

The general ARIMAX(p, d, q) model can be expressed as:

$$\Phi(L)(1-L)^d\, y_t \;=\; \Theta(L)\,\varepsilon_t \;+\; \boldsymbol{\beta}^\top \mathbf{x}_t,$$

where:

- y_t is the California retail gas price at week t,
- L is the lag operator ($Ly_t = y_{t-1}$),
- $(1-L)^d$ is the differencing operator of order d,
- $\Phi(L) = 1 - \phi_1 L - \phi_2 L^2 - \cdots - \phi_p L^p$ is the autoregressive polynomial of order p,
- $\Theta(L) = 1 + \theta_1 L + \theta_2 L^2 + \cdots + \theta_q L^q$ is the moving average polynomial of order q,
- $\varepsilon_t \sim \mathrm{WN}(0, \sigma^2)$ is white noise,
- $\mathbf{x}_t = [\text{wti_price, padd5_inventory}, \ldots, \text{product_supplied}]^\top$,
- $\boldsymbol{\beta}$ is the coefficient vector for the exogenous variables.

4.1.2 Order Selection

Earlier we plotted time series data. Observing the plot, although gas fluctuates, it increases over time. In recent years and following the COVID-19 pandemic, it caused massive economic and social changes even

more so. After we plotted ACF, PACF to help display our original data, as it decayed slowly, it indicated to us to take a difference.

ACF/PACF Diagnostics. After first differencing, we examine the autocorrelation function (ACF) and partial autocorrelation function (PACF) of the differenced training series. The ACF shows a sharp cutoff after lag 1, suggesting $q \leq 2$, while the PACF exhibits a similar cutoff pattern, suggesting $p \leq 2$. These diagnostics guide the bounds of our grid search.

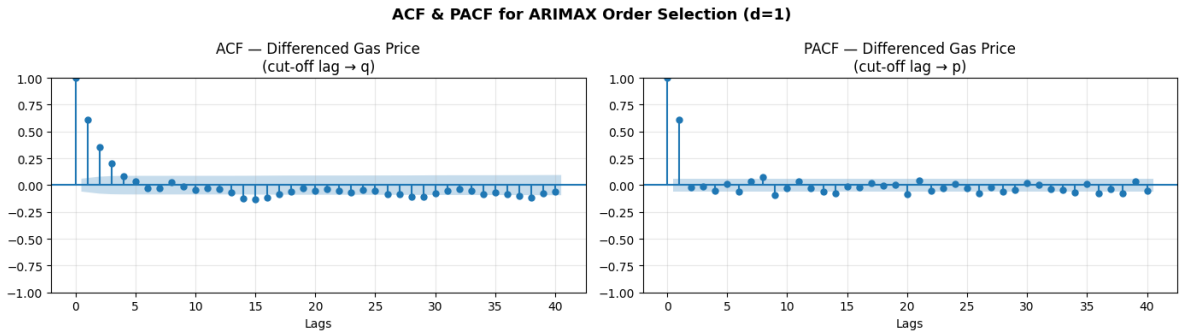

Figure 3: ACF and PACF of the differenced training gas price series with 95% confidence bands.

Stationarity Testing. We apply the Augmented Dickey–Fuller (ADF) test to the training-set gas price series. The test yields an ADF statistic of **-2.4515** with a p-value of **0.1277**, which fails to reject the null hypothesis of a unit root at the 5% significance level. This confirms that the gas price series is non-stationary in levels, requiring first differencing ($d = 1$).

Grid Search. To find the best model, we performed an exhaustive search over $p \in \{1, 2\}$, $d \in \{1\}$, $q \in \{1, 2\}$, yielding 4 candidate configurations. Each model was fit on the training set (1,074 observations) using maximum likelihood estimation via the `statsmodels.SARIMAX` implementation. Models producing validation predictions exceeding \$100/gallon in absolute value are discarded as unstable.

The best configuration by validation RMSE is **ARIMAX(1,1,1)** with:

- **AIC** $= -3{,}182.11$ (lowest among all candidates),
- **12-step validation RMSE** $=$ \$0.2056.

The full grid search results are shown in Table 5.

Table 5: ARIMAX order search results (validation).

Order (p, d, q)	**AIC**	**Val RMSE**
(1,1,1)	-3,182.11	0.2056
(2,1,1)	-3,180.49	0.2058
(1,1,2)	-3,175.95	0.2061
(2,1,2)	-3,172.00	0.2081

The results show that while configurations perform similarly, we favor the (1,1,1) specification.

4.1.3 Rolling Evaluation Protocol

For test-set evaluation, we employed a **rolling one-step-ahead** protocol with frozen parameters:

1. Fit the ARIMAX(1,1,1) model once on the training set of 1,074 observations.
2. Fast-forward the model state through the validation period (134 steps) without refitting, using `append(y, exog, refit=False)` to update the Kalman filter state.

3. At each test step $t \in \{1, \ldots, 135\}$, we forecast one step ahead using the true exogenous values, record the prediction, then append the true observation to update model state still without refitting.

This is a scenario where the model's parameters are fixed after initial training, but its state evolves as new data arrives. So, by test step 135, our ARIMAX model has seen all observations and its state reflects the full history, while our later GRU/LSTM models will still only see the last 12 weeks. We should note this subtle advantage in our ARIMAX model as its a stronger baseline than it might first appear.

4.2 Advanced Models: Recurrent Neural Networks

We also employed RNNS in addition to our ARIMAX models for comparison.

4.2.1 Vanilla RNN

The simplest recurrent architecture, the Vanilla RNN, processes sequential input through a hidden state that is updated at each time step:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{ih}\mathbf{x}_t + \mathbf{b}_{ih} + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_{hh}),$$

where $\mathbf{x}_t \in \mathbb{R}^5$ is the input vector at time t, $\mathbf{h}_t \in \mathbb{R}^{128}$ is the hidden state, and $\mathbf{W}_{ih}, \mathbf{W}_{hh}$ are learnable weight matrices. The output at the final time step is passed through a fully connected layer:

$$\hat{y} = \mathbf{W}_{fc}\mathbf{h}_T + \mathbf{b}_{fc},$$

where $T = 12$ is the sequence length, our lookback window. However, often the Vanilla RNN suffers from the **vanishing gradient problem**: during backpropagation through time (BPTT), gradients are repeatedly multiplied by the recurrent weight matrix $\mathbf{W}_{hh}$, causing them to decay exponentially and limiting the model's ability to learn longer-range dependencies. We tested two variants:

- **Vanilla RNN (0.1 dropout):** applies 10% dropout between the two recurrent layers for mild regularization.
- **Vanilla RNN (no dropout):** no dropout, allowing the model to fully utilize its capacity.

Both variants use the same **50,433 trainable parameters**.

As LSTM and GRUs use gating mechanisms to protect gradients from decaying, we employed those models as well.

4.2.2 Long Short-Term Memory (LSTM)

The LSTM (Hochreiter & Schmidhuber, 1997) addresses the vanishing gradient problem by introducing a **cell state** $\mathbf{c}_t$ and three multiplicative **gates** that regulate information flow. The structure follows:

Forget gate (decides what to discard from cell state): $\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$
Input gate (decides what new information to store): $\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$
Candidate cell state: $\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$
Cell state update: $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$
Output gate (decides what to output from cell state): $\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$
Hidden state: $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$

Note σ is the sigmoid function and $\odot$ denotes element-wise multiplication.

In LSTM, the cell state acts as a "conveyor belt" that allows gradients to flow across many time steps with minimal modification. Our LSTM uses **201,345 trainable parameters** which is approximately 4× the Vanilla RNN due to the four sets of weight matrices required by gating.

4.2.3 Gated Recurrent Unit (GRU)

The GRU (Cho et al., 2014) [3] simplifies the LSTM by merging the cell state and hidden state and reducing the number of gates from three to two. The structure follows:

Reset gate (controls how much of the previous hidden state to forget): $\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r)$
Update gate (balances between previous and candidate hidden state): $\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z)$

Candidate hidden state: $\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h)$
Hidden state update: $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$

With two gates instead of three, the GRU requires **151,041 parameters**, which is about 25% fewer than the LSTM while maintaining the ability to capture long term dependencies. The update gate $\mathbf{z}_t$ simultaneously controls forgetting and updating, acting as a combined forget/input gate.

4.3 Architecture Summary

All four of our RNN variants share the following architectural design:

Table 6: Shared RNN architecture across variants.

Component	**Specification**
Input dimension	5 features
Recurrent layers	2 stacked layers
Hidden size	128 units per layer
Output layer	Fully connected, 128 $\rightarrow$ 1
Sequence length	12 weeks (lookback window)
Prediction task	One-step-ahead (next week's gas price)

The input at each time step is a 5-dimensional vector of normalized features. The model processes 12 consecutive weekly observations and outputs a single scalar prediction for the next week.

4.4 Optimization & Training

Loss Function. We use **Mean Squared Error (MSE)** as the training objective:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2.$$

MSE is the natural choice for regression tasks and is directly related to our primary evaluation metric, with RMSE just being $= \sqrt{\text{MSE}}$.

Optimizer. We used an **Adam optimizer** with a learning rate of $\eta = 0.001$. Adam combines per-parameter adaptive learning rates with exponentially weighted gradient history, so its well-suited for the noisy gradients that's typically seen with recurrent network training.

Learning Rate Scheduling. We used a `ReduceLROnPlateau` scheduler as it halves the learning rate (factor = 0.5) after 5 consecutive epochs without validation loss improvement.

Early Stopping. We employed early stopping in which training terminates if the validation loss does not improve for 10 consecutive epochs (patience = 10). The model state corresponding to the lowest validation loss is restored after training completes, reducing overfitting while allowing time for convergence.

Sequence Construction. Sliding windows of length 12 are constructed independently within each split:

- **Train:** windows built from the 1,074 training observations $\rightarrow$ 1,062 sequences.
- **Validation:** the last 12 training observations are prepended as context, yielding 134 sequences.
- **Test:** the last 12 validation observations are prepended as context, yielding 135 sequences.

This ensures no future information leaks into training sequences while providing each validation/test sequence with appropriate historical context.

5 Experimental Setup

Table 7: Hardware and software configuration.

Component	**Specification**
Hardware	
CPU	Intel Xeon @ 2.20GHz
GPU	None — CPU only
RAM	12.7 GB (Colab session allocation)
OS	Ubuntu 22.04 (Google Colab, kernel 6.6.113)
Software	
Python	3.12.13
PyTorch	2.10.0 (CPU build)
NumPy	2.0.2
pandas	2.2.2
scikit-learn	1.6.1
statsmodels	0.14.6

5.1 Computational Times:

Table 8: RNN training times (CPU).

Model	**Total Time**	**Epochs**	**Avg. Time/Epoch**	**Parameters**
LSTM (no dropout)	29.05 s	26	∼1.12 s	201,345
GRU (no dropout)	55.01 s	50	∼1.10 s	151,041
Vanilla RNN (0.1 dropout)	26.38 s	44	∼0.60 s	50,433
Vanilla RNN (no dropout)	25.07 s	50	∼0.50 s	50,433

Key observations:

- The **LSTM** completed in only 26 epochs, triggering early stopping relatively quickly. Despite having the most parameters, its per-epoch time (∼1.12 s) is similar to the GRU due to the optimized PyTorch LSTM kernel.
- The **GRU** ran for all 50 epochs without early stopping, indicating continued improvement throughout the training. Thus, it made sense it's total clock time of 55.01 s was the highest among all models.
- Both **Vanilla RNN** variants are approximately 2× faster per epoch than the gated architectures, reflecting their simpler structure of only one recurrent transformation versus three or four in the gated units.

5.1.1 ARIMAX Training Time

The ARIMAX model fit in under 1 second using MLE. The grid search over 4 configurations completed in approximately 2–5 seconds. The rolling one-step-ahead evaluation on 135 test steps took approximately 10–30 seconds due to sequential Kalman filter updates.

5.1.2 Total Project Compute

Including the GRU ablation study (9 configurations of approximately 30s – 1 minute each), additional hyperparameter experiments, and iterative development, we estimate a total CPU compute budget of approximately **2 CPU-hours** for the project. This is relatively efficient compared to other deep learning

projects, as we ran only ARIMAX, Vanilla RNNS, LSTM, and GRU and reflects our relatively smaller data scale and CPU-only computation.

5.2 Hyperparameters

Table 9 summarizes the hyperparameters used in the final models.

Table 9: Final hyperparameter settings.

Hyperparameter	**Value**	**Justification**
Lookback Window	12 weeks	About 3 months of context.
Batch size	32	Standard default; stable gradient estimates for a small dataset.
Max epochs	50	With early stopping (patience=10), prevents under- and over-training.
Early stopping patience	10	Allows recovery after learning-rate reductions.
Hidden size	128	Selected based on ablation study (see Section 7.2).
Number of layers	2	Captures hierarchical temporal features.
Learning rate	0.001	Default Optimizer; reduced by scheduler on plateaus.
LR scheduler	ReduceLROnPlateau	Factor=0.5, patience=5 epochs; adapts to convergence speed.
Dropout	[0.0, 0.1, 0.2]	Used to evaluate regularization effects on model capacity.
Random seed	41	Fixed for reproducibility.

5.3 Reproducibility

All random seeds are fixed at the beginning of the notebook (`torch.manual_seed(41)`, `np.random.seed(41)`) to ensure reproducibility. No data shuffling is used in the `DataLoader` (`shuffle=False`), preserving temporal ordering throughout training. The full codebase is available at the Github repository listed in the Codebase section.

6 Results

6.1 Model Performance Comparison

The following table shows the results of how every model did on the 135-week test set. We used 5 different models and evaluated them using the one-step-ahead method, trying to predict a week ahead rather than an indefinite amount of time. The 135 weeks spans from approximately December 2023 to February 2026.

Table 10: Test performance comparison.

Model	**RMSE**	**MAE**	R^2	**PCC**	**Parameters**
ARIMAX(1,1,1)	**0.0701**	**0.0491**	**0.9616**	**0.9812**	-
GRU (no dropout)	0.0864	0.0680	0.9417	0.9762	151,041
Vanilla RNN (no dropout)	0.1030	0.0806	0.9172	0.9653	50,433
Vanilla RNN (0.1 dropout)	0.1142	0.0873	0.8982	0.9578	50,433
LSTM (no dropout)	0.1876	0.1491	0.7253	0.8569	201,345

Key Findings.

- The main takeaway is that **the ARIMAX model was the best; even outperforming the more complex neural network models**. It had the lowest RMSE and MAE, the highest R^2, and highest PCC. Therefore, it was the most accurate model, and had extremely strong directional tracking.
- The next area of focus has to do with the neural network models we tested. **GRU was the best deep learning model**. It had the lowest RMSE and MAE, closely behind the ARIMAX model,

and highest R^2 and PCC. The two Vanilla RNN models were a close 2nd and 3rd across all metrics, and the LSTM model was the clear worst. This is especially interesting since the GRU model had approximately 50,000 fewer parameters than the LSTM model, and performed significantly better. Since GRU and both Vanilla RNNs performed better than the LSTM, this indicates that the extra complexity of the LSTM is actually hurting it, likely causing overfitting because of the relatively small dataset.

- If we take a closer look at the Vanilla RNNs, we can see that **dropout hurts performance.** The model with no dropout regularization was better across the board compared to the Vanilla RNN with 0.1 dropout regularization. This might also be due to the relatively small training set.
- Just to reiterate and dive deeper into potentially the most interesting finding, **LSTM underfits despite having the most parameters.** Since early stopping took place, it indicates that the model struggled to converge and that loss plateaued pretty quickly.

6.2 Learning Curves

From these training vs validation loss curves, we can see different optimization patterns for the three models. First,

Figure 4: Training and validation loss curves for GRU, LSTM, and Vanilla RNN (no dropout).

- **GRU:** Training and validation loss consistently decreased over all the epochs. Early stopping did not take effect, and there were no signs of divergence. There is no significant gap between the training and validation loss, which does not give us any reason to suspect overfitting.
- **LSTM:** Similar to the GRU, it starts with a rapid drop in validation loss but plateaus around 15 epochs. A gap between the validation and training loss starts to appear right before early stopping takes place. This helps explain why it did poorly on the test set, and that the model was likely underfitting.
- **Vanilla RNN:** The curves look similar to the GRU graph. Training makes it all the way to 50 epochs with steady improvements the entire time. Validation and training loss decreases nearly the entire time.

6.3 Forecast Visualization

The forecast comparison plot overlays all model predictions against actual gas prices for the 135-week test period:

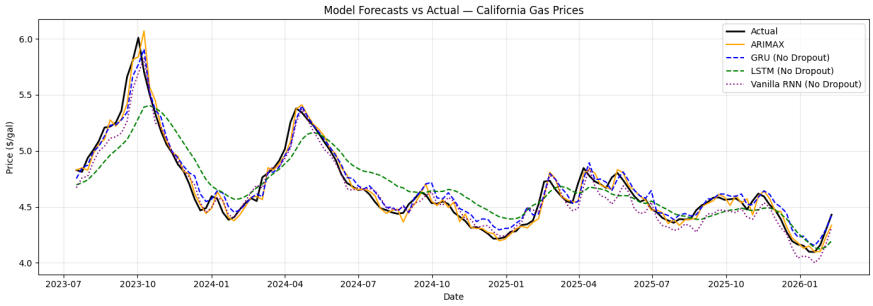

Figure 5: Model forecasts versus actual California gas prices on the test period (135 weeks).

From the plot, we can confirm our findings about each of the models. ARIMAX was the best, and we can see that it followed the general trends and spikes extremely well. GRU captured the basic trends but struggled with sudden spikes. The Vanilla RNN was similar to the GRU model, but suffered from lag a bit more and underestimated the actual prices during spikes. Lastly, we can see why LSTM had such a bad test performance. It shows very consistent smoothing over the short time period which is clear underfitting

6.4 Bar Chart Comparison

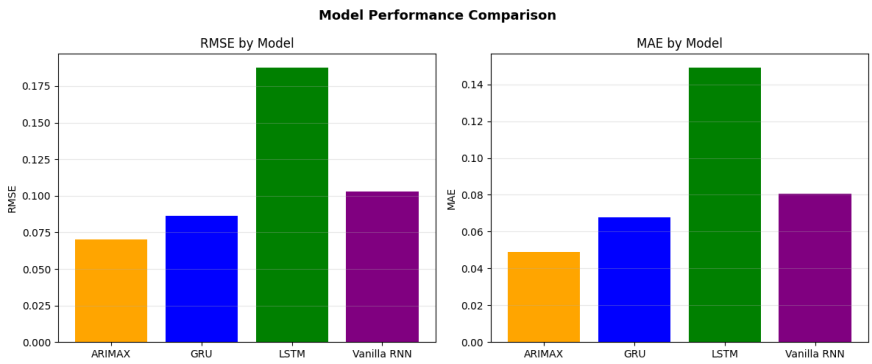

Figure 6: RMSE and MAE comparison across models.

Here is a visual that better puts into context how much better ARIMAX was than the rest, and how bad the LSTM model actually did. Also, that GRU and Vanilla RNN were not too far behind the ARIMAX model.

6.5 ARIMAX Residual Diagnostics

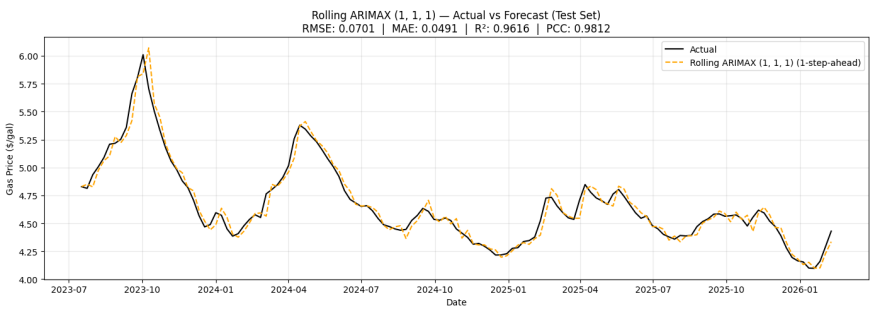

Figure 7: ARIMAX residual diagnostics:

Table 11: ARIMAX residual summary statistics.

Statistic	Value
Mean	-0.0005
Standard deviation	0.0701
Skewness	-0.1885
Excess kurtosis	5.4668
Durbin–Watson	1.8915
Jarque–Bera p-value	0.0000

Residual Summary Statistics. Based on the residual diagnostics, the ARIMA model does not show any bias with a mean residual of approximately 0. The Durbin-Watson statistic of 1.89 suggests minimal first-order autocorrelation and indicates that the model adequately captures linear dependence. But, we need to note that the Jarque-Beta test rejects normality because of the Excess kurtosis value of approximately 5.47. This indicates that the trail is heavier than a usual Gaussian distribution. Overall, these values imply that the large forecast errors are happening more often than we would usually expect under the normal assumptions.

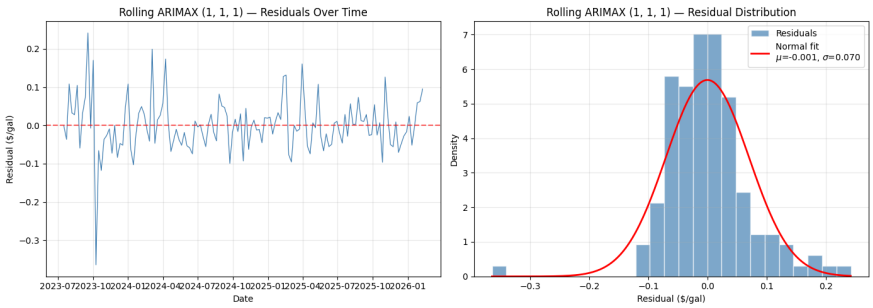

Figure 8: ACF and PACF of ARIMAX residuals (50 lags) with 95% confidence bands.

These plots reveal that almost all of the autocorrelations fall inside the 95% confidence interval. This confirms that the ARIMAX model captured the dominant linear temporal dependence.

7 Discussion, Ablations, & Error Analysis

7.1 Discussion: Why Does ARIMAX Outperform Deep Learning?

Despite being the simplest (fewest parameters) model, ARIMAX was the best. There are a few reasons why this can happen. One reason was that regarding time series, it is a relatively small dataset, at least for deep learning models. Deep learning models are most useful with an immense amount of data to learn from. The parameter-to-sample ratio is extremely high for the LSTM model, especially compared to the ARIMAX model, which only uses 9 free parameters. By the bias-variance tradeoff, simple models can be more effective than deep learning models when there is not a lot of data available to train on. Another reason is because of the linearity in the data-generating process. We can see that WTI crude oil price is the most dominant predictor with a fairly high correlation between the two. The formula to get gas price is fairly straightforward and is heavily driven by the crude oil price, which is something that simple models can easily capture. Especially because this relationship is practically linear. On the other hand, RNNs have to work much harder to learn these relationships. Also, it was due in part to our one-step-ahead evaluation method. This method benefits models that weigh the most recent observations the most. The ARIMAX model integrated new data very easily, while the RNN models used fixed weights that didn't update during the evaluation period.

7.2 Ablation Study: GRU Architecture Sensitivity

We conducted an ablation study on the GRU model to see how the different hyperparameters effect model performance. In order to draw causal conclusions, we changed one hyperparameter value at a time while keeping the others constant. Assuming that there is no interaction effect going on between the hyperparameter values. Our goal was to find which values led to the best model performance.

7.2.1 Hidden Size

Table 12: GRU ablation: hidden size.

Hidden Size	RMSE	MAE
32	0.1834	0.1508
64	0.1238	0.0979
128	**0.0942**	**0.0718**

So, increasing hidden state size significantly improved model performance. Changing the hidden state size from 32 to 128 saw a 49% reduction in RMSE. This tells us that GRU benefits more from extra capacity.

7.2.2 Dropout Rate

Table 13: GRU ablation: dropout rate.

Dropout	RMSE	MAE
0.0	**0.0942**	**0.0718**
0.1	0.1021	0.0794
0.2	0.1659	0.1276

While the differences here are less significant than the changes in hidden size, there is still a trend. No regularization resulted in the best model, and increasing the dropout percentage steadily made the model worse. This tells us that there is not ejnough data for dropout regularization to be beneficial, because reducing capacity is harmful as we just learned.

7.2.3 Lookback Window

Table 14: GRU ablation: lookback window length.

Lookback (weeks)	RMSE	MAE
12	0.0942	0.0718
25	**0.0927**	**0.0716**
52	0.0950	0.0766

While we found that optimal lookback window to be 25 weeks, the change in performance of different lookback windows is minimal. This can tell us that gas prices are primarily driven by medium-term factors rather than looking too far in the past.

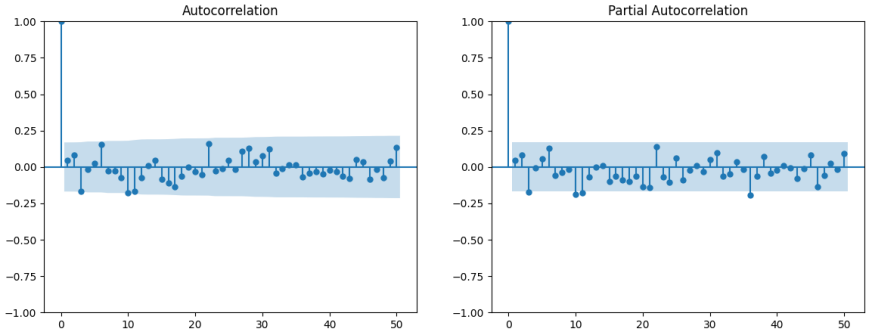

Figure 9: GRU ablation study summary table: rolling one-step-ahead test performance.

7.3 Error Analysis

Table 15: Final test-step predictions.

Model	Predicted	Actual	Error
ARIMAX	4.3348	4.4300	-0.0952
LSTM	4.1961	4.4300	-0.2339
GRU	4.4257	4.4300	-0.0043
Vanilla RNN (no dropout)	4.3170	4.4300	-0.1130

At the last test step, every model underestimated the actual price. But, GRU actually made the best prediction while the LSTM made the worst one. This aligns with the fact that LSTM had too much smoothing in its predictions.

7.3.1 Sources of Error

There were a few scenarios that we can point out from when our model struggled the most. First, all of the models under predict rapid spikes. In general, it is extremely hard to anticipate such rapid movements from the data we have without any context of world events which are the leading drivers of spikes. Also, it is difficult for the models to understand an event like COVID-19 which had quite the recovery period, and may have caused long-lasting effects that historical patterns would not be able to predict. Lastly, since we are looking at California only, there were supply shortages that would not necessarily align with national trends. So some of our covariates might not fully capture what we are trying to capture.

7.4 Interpretation

Overall, the results show that classical models can compete with deep learning methods for time-series forecasting if the dataset is particularly small, predictor relationships are approximately linear, and when using one-step-ahead evaluations. It is also important to note GRU's success. It tells us that the two-gate architecture was sufficient in this scenario. Combine that with the LSTM's struggles, reveals that a more modest dataset will not necessarily benefit from additional complexity.

8 Conclusion & Future Work

8.1 Summary of Findings

We compared 5 forecasting models to predict weekly retail gas prices in California, and evaluated the models on a 135-week test set.

- ARIMAX was the best performing model by all metrics we considered. It had the lowest RMSE and MAE at 0.07 and 0.05 respectively. Along with a R^2 of 0.96 which is extremely high
- GRU was the best performing deep learning model. It was slightly better than the two Vanilla RNNs but significantly better than the LSTM model. It proved to be a good balance of complexity and capacity.
- From the ranking of the deep learning models, we can conclude that additional complexity does not guarantee an improvement in predictive performance. The LSTM model had the far most parameters, but clear worst performance.
- Dropout regularization negatively affected the model in this scenario. Also when we applied it and increased the percentage, it increasingly hurt the model performance more and more. GRU cares more about capacity over anything else
- From the ablation study, we also learned that increasing hidden state size improved performance, while adjusting the lookback window had negligible differences.

8.2 Limitations

- Since we only used California data, we cannot generalize these findings when predicting gas prices of other states. Since there are unique factors about California that might not apply to other states.
- The covariates can be helpful in bringing additional information, but they are not California specific and might not necessarily tell the full story.
- In general, we have a smaller dataset for typical deep learning problems which puts the more complex models at an inherent disadvantage.

8.3 Future Work

1. Explore a Dual-Stage Attention RNN, which may better capture features and temporal dependencies in gas prices.
2. Incorporate policy and macroeconomic features such as carbon prices and regulatory changes relevant to California and gas prices.
3. Expand the analysis to regional variation within California, fe. examining price dynamics across cities in California.

8.3.1 Extended Study: Further Timeline, Larger Computational Resources, and Adaptation for a Thesis

With additional time and computational resources, to specify in addition to the bullet points mentioned above, we would further adjust our modeling and implement RNN fine-tuning for a fairer comparison with ARIMAX, and run a Transformer architecture search (fe. Temporal Fusion Transformer) via Bayesian optimization. Furthermore, a thesis-level extension would include multi-step-ahead forecasting fe. ($h = 4, 8, 12$ weeks), as well as incorporate further modeling such as an ARIMAX-LSTM model or DA-RNN. On the data side, we would move into incorporating more California-specific variables like LCFS credit prices. Also, we would validate our findings on a second high-regulation market.

References

[1] Christiane Baumeister and Lutz Kilian. Forecasting the real price of oil in a changing world: A forecast combination approach. *Journal of Business & Economic Statistics*, 33(3):338–351, 2015.

[2] Indradeep Chatterjee. Cyclic encoding in machine learning: A practical guide with code and intuition. `https://medium.com/@indradeep.chatterjee/cyclic-encoding-in-machine-learning-a-practical-guide-with-code-and-intuition-049846419ccb`, 2025. Medium article.

[3] Kyunghyun et. al Cho. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734. Association for Computational Linguistics, 2014.

[4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[5] Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The M4 competition: Results, findings, conclusion and way forward. *International Journal of Forecasting*, 34(4):802–808, 2018.

[6] U.S. Energy Information Administration. California state energy profile: Analysis. `https://www.eia.gov/states/CA/analysis`, 2026. Accessed: 2026-03-17.

[7] U.S. Energy Information Administration. Crude oil prices: West texas intermediate (wti) - cushing, oklahoma [dcoilwtico]. Retrieved from Federal Reserve Bank of St. Louis, FRED: `https://fred.stlouisfed.org/series/DCOILWTICO`, 2026. Accessed: 2026-03-17.

[8] U.S. Energy Information Administration. U.s. energy data and statistics. `https://www.eia.gov/`, 2026. Accessed: 2026-03-17.

Team Contributions

All team members participated in project meetings, peer review of each other's work, and co-authorship of the final written report. Individual primary responsibilities are listed below.

- **Kathleen Wu:** Data pipeline & EDA; EIA data collection, merging, and preprocessing; GRU ablation study (hidden size, dropout, lookback window.
- **Antoinette Nguyen:** ARIMAX model specification; ADF stationarity testing; grid search order selection; residual diagnostics.
- **Nick Wong:** RNN architecture implementation (Vanilla RNN, LSTM, GRU) in PyTorch; training pipeline (Adam optimizer, LR scheduler, early stopping).
- **All Members:** Results interpretation, error analysis, introduction, conclusion, related work.

Codebase

The full project codebase is available at:

`https://github.com/Kathleen-Wu/Retail-Gas-Price-Forecasting/tree/main`